

System Design

MLE

Content

- MLE System Design 簡介
- ML Pipeline Design
- Building the ML Pipeline
 - 最有效練習: Chatgpt
- ML General
- 焦點ML Design Questions
- Traditional System Design 簡介
- Building the System
 - 最有效練習: Chatgpt

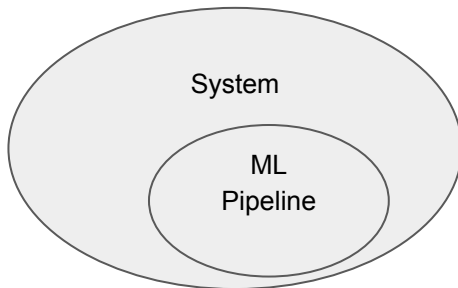
MLE System Design 簡介

ML System Design

MLE / DS

Traditional System
Design

Software Eng



在 MLE 的面試中, Interviewer會根據需求選擇其中一類, 或是將兩者 **結合**, 讓你展示如何將 ML Pipeline整合進一個穩定的 System

- 這意味著你不僅需要了解 **ML Pipeline**, 還需要 Prepare **Distributed System Design**

PainPoint:

Coding可以操Leetcode

System Design 太空泛 無從入手

ML vs Software System Design

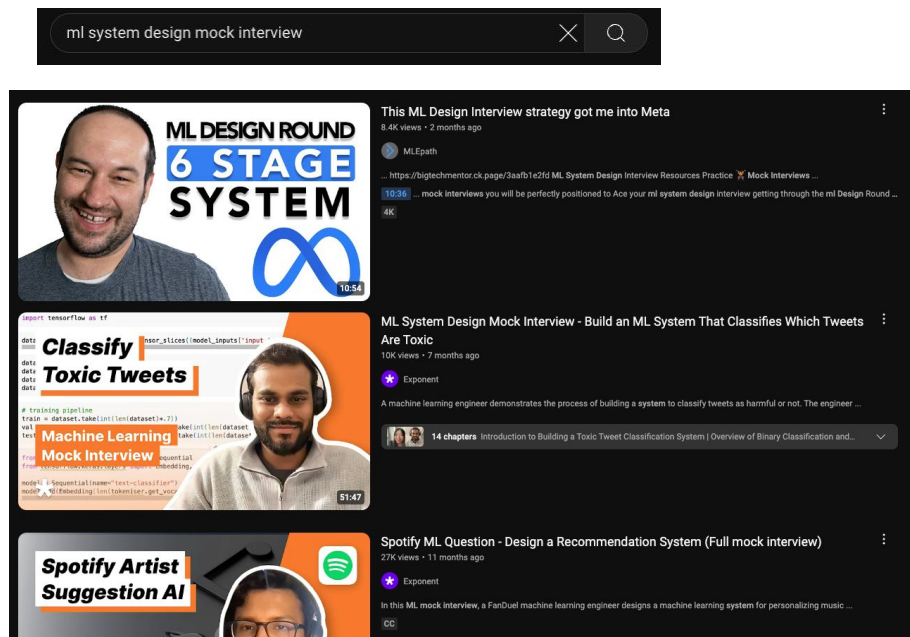
	ML System Design	Traditional System Design
Objective	Focus on Data flow, features, model development, deployment, and inference	Design stable and scalable systems and infrastructure
Technical Scope	ML Pipeline, model inference, data processing	API design, microservices, database optimization
Considerations	Model performance, data drift, automation	High availability, low latency, fault tolerance
Common Areas	System performance optimization, monitoring, and maintenance	System must support workloads for model inference

ML Pipeline Design

Design a ML Pipeline

- 這部分針對如何設計 **包含ML model**的system, 重點在於處理 **data、model development、training、productionalization**
- 如何配合ML去解決問題 → 解決問題千萬種方法, Why ML?
 - 觀察, 分析, 佈局
 - Always have 2 sides:
 - ML side
 - System side
- Interviewer主要評估你如何 Design End to End 的 ML Pipeline, 包括Data Handling、Modelling and Production
 - E.g. Recommendation System
 - ML side: Data, Model, Evaluation
 - System side: Scalability, Latency
- 當中會問ML General 問題
 - Boosting vs Bagging
 - Recall vs Precision

Youtube - Interview 應用



- 大概感受
- 整理重點
- 學習溝通方式
- Interviewer 提出什麼問題？

https://www.youtube.com/results?search_query=ml+system+design+mock+interview

Step by Step Building the ML Pipeline

Major Steps

1. **Problem Understanding**
2. **Goals Identification**
3. **Data Pipeline**
4. **Model Pipeline**
5. **Model Deployment**
6. **Monitoring and Maintenance**

Minor Considerations

- **Scalability and Optimization**
- **Continuous Integration and Continuous Deployment (CI/CD)**
- **Bonus: Collaboration with Other Teams**

Major Step 1 - Problem Understanding

- 問問題
 - Clarify the Problem Statement
 - Business or User Requirements
- Example: Design youtube
 - Youtube太多components, 需要問題去Narrow down the scope
 - Search Function / Video Streaming / Optimize the recommendation etc.?
 - 每一個獨立出來都可以做
 - 需跟Interviewer confirm做which component
 - Requirements
 - Who are the end-users, and what is their context?
 - System Requirements (Performance? Scalability?)

Major Step 2 - Goals Identification

- Business Goals
 - 普通人: Ask what the business or user goals are
 - Are we optimizing for user engagement, retention, revenue, or something else?
 - 高手: 整合 Information → Make Assumptions
 - 腦海整理好現有的故事, 嘗試 Define business goals 再ask for confirmation
 - 引導Interviewer去你熟識既領域
 - Metrics to measure success
 - User engagement, Retention, Revenue
- ML Goals (Support Business Goals)
 - Always under business goals → 如何利用ML去達到Business Goals
 - 把問題Frame為一個簡單的ML Solution
 - Regression
 - Binary Classification (Probability)

Major Step 3 - Data Pipeline

- 萬事由Data出發
- 所有Event / Decision (包括Model) 都要Tailor-made from data
- 四個步驟Cover 所有Data相關要點
 1. Data Collection
 - Design methods to collect raw data from multiple sources (e.g., databases, APIs, streaming data)
 2. Data Storage
 - Choose appropriate storage solutions (SQL, NoSQL, data lakes)
 - Based on data volume, latency, and access needs
 3. Data Cleaning & Preprocessing
 - Standardize steps for handling missing data, outliers, and normalization
 4. Feature Engineering
 - Design a feature store where engineered features are stored and can be reused for both training and inference
- During Interview: Make Assumptions / Clarifications on collected dataset
 - 直接影響後續處理
 - Dataset 大小 / Completeness / Timeliness / Imbalance etc.

Major Step 4 - Model Pipeline

- 根據 Defined ML Goal, 配合 Data Assumption 去建立 Model
 - 先 Clarify 現有的 Resource
 - GPUs, hardwares, time, human resources
 - 永遠都有 Best Solution / 只有 Optimal Solution based on current situation
 - E.g. Smaller Dataset + 追求 Performance + OK Resource → Tree based model
 - E.g. Large Enough Dataset + 追求 Performance + Good Resource → Neural Network
- 要點
 - Model Selection & Training
 - Select the model from a pool of models (Comparison between models)
 - Implement a pipeline for training models
 - May include automated hyperparameter tuning, model validation, and selection
 - Experiment Tracking
 - 用相應 ML Metrics 去 measure performance (RECALL vs PRECISION etc.)
 - Use tools to keep track of different model versions, hyperparameters, metrics, and datasets
 - E.g. MLflow, Weights & Biases
 - Version Control
 - Store model versions with associated metadata (e.g., data used, hyperparameters) for reproducibility and rollback

Major Step 5 - Model Deployment

- 已經Train 好的Model放到Production
 - 先confirm inference的限制 (Always on? Downtime? Resources)
 - Data 進來的format
 - 萬用: Json format, 可describe how the json looks
- Deployment Strategies
 - Batch Processing: Suitable for less time-sensitive applications, where predictions are generated periodically (e.g., daily recommendation updates)
 - Real-Time Inference: Low-latency setup for applications requiring instant responses, such as fraud detection or search recommendations.
- Deployment Patterns:
 - A/B Testing: Deploy multiple models simultaneously to compare performance with live traffic.
 - Rolling Deployments: Gradually shift traffic to a new model to minimize risks.
 - Blue-Green Deployments: Keep two identical environments and route traffic between them for seamless updates.

Major Step 6 - Monitoring and Maintenance

- 只需/盡量輕輕帶過
- Model Performance Monitoring
 - Track metrics such as accuracy, latency, and throughput in production to ensure model performance remains optimal
- Data Drift and Concept Drift
 - Implement mechanisms to detect changes in data distribution or model predictions over time, triggering retraining or other corrective actions
- Alerting and Logging
 - Set up alerts for metric drops, drift detection, and logging for traceability in case of failures

Minor Considerations

Scalability and Optimization

- **Load Balancing:** Distribute prediction requests across multiple instances to handle high traffic efficiently.
- **Caching:** Use caching strategies (e.g., Redis, Memcached) for frequently requested predictions to reduce latency.
- **Model Optimization:** Apply model compression techniques, such as quantization and pruning, to reduce model size and improve inference speed.
- **Resource Management:** Design autoscaling solutions to allocate resources dynamically based on workload.

Continuous Integration and Continuous Deployment (CI/CD)

- **Automated Testing:** Build automated pipelines that test models for accuracy, drift, and latency before deployment.
- **Retraining and Re-deployment:** Automate retraining of models based on new data or degradation in performance, and set up pipelines for seamless re-deployment.

Bonus: Collaboration with Other Teams

- Work closely with **Data Engineering** for data pipeline management, **DevOps** for infrastructure setup, and **Product Teams** to align model performance with business metrics.

最有效練習 : Chatgpt Building the ML Pipeline

實際操作 : Chatgpt

Lets say we are having an ML design interview, **design youtube**

實際操作 : Prompt Framework

Problem Understanding: 問問題

Let's say we are now in an ML engineer interview. I'll ask you a question about ML system design.

The question is: {}

*After you receive the question, do not start to answer, I'll prompt you step by step,
For each step, give me as many details and explanations as possible.*

Before jumping into any solution, start by asking some questions to understand the problem (Keep this clean, not too many questions) and understand the scale and size of the problem. And also the system requirements

During Interview: Expect 去問 Interviewer 問題, Clarify 題目, Focuses

- Jot down the notes/feedbacks for next part

Design Youtube

**Problem
Understanding**

**Business
Goals**

Data Pipeline

**Feature
Engineering**

**Model Development
and Evaluation**

Deployment

Let's say now we are in a ML engineer interview, I'll give you a question of ML system design.

The question is: Design Youtube

After you receive the question, do not start to ans, I'll prompt you step by step,
For each step, give me all the details and explanations as much as possible.

Before jumping into any solution. Start by asking some questions to understand the problem (Keep this clean, not too many questions) and understand the scale and size of the problem. And also the system requirements



Got it! Here's how I'd start by asking clarifying questions to understand the problem and gather requirements for designing YouTube:

Questions to Understand the Problem:

1. **Core Purpose:** What is the main goal of this system? Is it primarily for video playback, content recommendations, user engagement, or all of these?
2. **Scale and Size:**
 - How many daily active users are we designing for?
 - How much video content is uploaded daily?

Questions to Understand the Problem:

1. **Core Purpose:** What is the main goal of this system? Is it primarily for video playback, content recommendations, user engagement, or all of these?
2. **Scale and Size:**
 - How many daily active users are we designing for?
 - How much video content is uploaded daily?
 - What is the average duration of user sessions?
 - What is the expected peak traffic (e.g., during live events)?
3. **Key Functional Requirements:**
 - What features are essential? (e.g., video uploading, streaming, recommendations, search, user profiles, comments, and notifications.)
 - Should the system support real-time features like live streaming or chat?
 - How important is personalization for users?
4. **Content Recommendations:**
 - How should the recommendation system work? (e.g., based on user behavior, trends, subscriptions, or categories.)
 - How do we prioritize relevance, diversity, and recency in recommendations?
5. **Performance and Latency:**
 - What are the performance requirements for video playback and recommendation updates?
 - What's the acceptable latency for v. load time and other interactions?

- 太多問題, 自選幾個
- 嘗試代入 Interviewer 角色, 回答問題
- 嘗試不同 Combination
 - Many users & recommendation focused & Performance focused
 - Many users & Search Engine & Latency focused

實際操作：Chatgpt 練習 - Prompt Framework

Business & ML Goals

Notes & Feedbacks from previous section +

Then, define the Business and ML Objectives (make an assumption about the ML problem—regression, classification, or another type) and the metrics, explaining why you chose them

- 前一part的feedbacks, e.g.
 - ~ 10 million of users per day
 - ~ 1 million videos per day

Design Youtube

Problem
Understanding

Business
Goals

Data Pipeline

Feature
Engineering

Model Development
and Evaluation

Deployment



Business Objectives:

1. **User Engagement:** Ensure users spend more time on the platform by providing relevant content.
2. **Content Discovery:** Help users discover videos they like while giving visibility to a diverse set of content creators.
3. **Creator Growth:** Encourage video creators to produce high-quality content by optimizing their reach and viewer engagement.
4. **Platform Revenue:** Increase revenue through targeted ads, premium subscriptions, and partnerships.

ML Objectives:

1. **Personalized Recommendations (Primary):** Provide a personalized home feed for each user, suggesting videos that maximize user engagement and satisfaction.
 - **Problem Type:** Multi-class classification or ranking problem.
 - Predict the likelihood of a user interacting with (e.g., clicking, watching, liking) a video and rank videos accordingly.
2. **Content Search and Discovery (Secondary):** Enable users to find content efficiently by optimizing search relevance.
 - **Problem Type:** Information retrieval and ranking problem.
 - Rank search results based on relevance to a user's query.

- Business Goal

- Interview時可選幾個
- Depends on Interviewer feedback

- ML → Supporting business goals

- Classification
- Ranking

實際操作: Chatgpt 練習 - Prompt Framework

Data Pipeline

During Interview: 先問 Interviewer: What is the size of the data that we can collect

- 大中細 datasets 直接影響處理方式

Then next step would be

Data Engineering: Identify and collect data sources

How can we build the ETL pipeline (In details)

This step should involve ensuring data quality, addressing balanced or imbalanced data (If applicable to the problem), maintaining consistency, and preparing it for analysis and modeling

Storage needs

Give some approaches to cleanse the data

After that, describe the data we have in the end (it can be a table, images or text, just choose if applicable)

Design Youtube

Problem
Understanding

Business
Goals

Data Pipeline

Feature
Engineering

Model Development
and Evaluation

Deployment

Then next step would be

Data Engineering: Identify and collect data sources

How can we build the ETL pipeline (In details)

This step should involve ensuring data quality, addressing balanced or imbalanced data (if applicable to the problem), maintaining consistency, and preparing it for analysis and modeling

Storage needs

Give some approaches to cleanse the data

After that, describe the data we have in the end (it can be a table, images or text, just choose if applicable)

Data Engineering: Identify and Collect Data Sources

To build YouTube, data engineering involves identifying data sources, creating an ETL pipeline, ensuring data quality, and preparing data for modeling and storage.

1. Identify Data Sources

User Behavior Data:

- **Clickstreams:** Logs of user actions like clicks, watch events, search queries, and likes.
- **Watch History:** Videos a user has watched, with timestamps and durations.
- **Engagement Metrics:** Likes, dislikes, shares, and comments.

Video Metadata:

- **Title, Description, and Tags:** Textual information for search and recommendations.
↓
- **Thumbnails:** Visual previews used in recommendations.

5. Final Dataset Description

The processed data is organized as follows:

User Interaction Table (Structured):

UserID	VideoID	WatchTime (s)	Click	Like	Dislike	CommentCount	Timestamp
12345	abc123	180	1	1	0	2	2024-12-01T10:00:00Z

Video Metadata Table (Structured):

VideoID	Title	Duration (s)	UploadTime	Tags	ThumbnailURL	Category
abc123	"Cooking Tips"	300	2024-11-30T08:00:00Z	["Cooking", "Food"]	https://.../thumb.jpg	"Lifestyle"

Thumbnails Dataset (Images):

- **Images:** Stored in an image database or blob storage, accessible via URLs.
- **Metadata:** Mapped to VideoID for reference.

Aggregated Features Table:

VideoID	Views	AvgWatchTime (s)	CTR	EngagementScore
abc123	10000	200	0.15	0.85

This dataset is now ready for analysis and machine learning. Would you like me to move forward with the modeling or discuss real-time updates to this pipeline?

必需知道dataset format & content

實際操作 : Chatgpt 練習 - Prompt Framework

Feature Engineering

Then next step would be

Feature Engineering: Develop features.

Feature selection is critical to building an effective model

List out the potential features and how you want to engineer and you can talk a little about where to store the features

Design Youtube

Problem
Understanding

Business
Goals

Data Pipeline

Feature
Engineering

Model Development
and Evaluation

Deployment

Feature Engineering for YouTube System Design

Feature engineering is critical for creating meaningful inputs for the machine learning models powering recommendations, search, and ads. Below is a breakdown of potential features, how to engineer them, and where they could be stored.

1. Potential Features

User Features

- **Demographic Data:**
 - **Age, Gender, Location:** Use for personalized recommendations and ad targeting.
 - **Engineering:** One-hot encode categorical features (e.g., gender, location).
- **Behavioral Data:**
 - **Watch History:** Encode the last N watched videos as a sequence.
 - **Engineering:** Use embeddings to represent watched videos.
 - **Session Duration:** Total time spent on the platform in the current session.
 - **Interaction Rates:** Calculate click-through, like, and comment rates for the user.
 - **Engineering:** Aggregate over a time window (e.g., past 7 days).
 - **Search History:** Tokenize and embed search terms.

Video Features

- **Content Data:**
 - **Metadata:** Title, description, tags, and categories.
 - **Engineering:** Preprocess text (e.g., tokenize, remove stop words) and use TF-IDF or word embeddings (e.g., Word2Vec, BERT).

Interaction Features:

- Create cross-features (e.g., interactions between user demographic and video category).

4. Final Feature List Example

Feature Name	Type	Description
User_Age	Numeric	Age of the user.
Video_Embedding	Vector	Pre-trained embedding of the video.
Session_Duration	Numeric	Total time spent in the current session.
CTR	Numeric	Click-through rate of the video.
Title_TF_IDF	Vector	TF-IDF vector for the video title.
Watch_History_Embeddings	Sequence	Sequence of embeddings of watched videos.
Timestamp_Cos_Sin	Cyclical	Hour of day converted to cyclical features.
Region_Popularity_Score	Numeric	Normalized engagement in user's region.
Device_Type_OneHot	One-hot	One-hot encoding for device type.

This comprehensive feature set is stored and indexed by user IDs, video IDs, and timestamps to support both training and inference.

較進階可以留意feature data type

Data Parts Done

Before Jumping into model selection right away, 問interviewer以下問題

- Computational resource we have against the performance we wanna achieve
- Do we have enough computational resources to train a complex model?
- Can i assume that we have the dataset size is enough to support a complex model training?
- Model Complexity vs. Efficiency? Interpretability?

Jot down 答案 帶到下part

實際操作: Chatgpt 練習 - Prompt Framework

Model Development and Evaluation

Notes & Feedbacks from previous section +

Model Development and Evaluation

What kinds of model we use to {Question}

List some potential models and describe them.

Compare potential models, choose an appropriate machine learning model.

Always go with a simple model first for benchmarking.

Then choose another primary model (Based on the current information you have)

- *Explain the model you choose*
- *Explain in details on how you wanna train the model and what metrics for model performance evaluation*
(No code, just explain is ok)

- 前一part的feedbacks, e.g.
 - High performance, assume we have enough computational power
 - We have a large dataset

Problem
Understanding

Business
Goals

Data Pipeline

Feature
Engineering

Model Development
and Evaluation

Deployment



High performance, assume we have enough computational power
We have a large dataset

Model Development and Evaluation

What kinds of model we use to {Question}

List some potential models and describe them.

Compare potential models, choose an appropriate machine learning model.

Always go with a simple model first for benchmarking.

Then choose another primary model (Based on the current information you have)

Explain the model you choose

Explain in details on how you wanna train the model and what metrics for model performance evaluation (No code, just explain is ok)



Model Development and Evaluation

Given the large dataset, sufficient computational power, and the problem of building a recommendation system for YouTube, we will focus on models suitable for **personalized video recommendations** and **search relevance**. Here's the step-by-step breakdown:

1. Potential Models

Simple Benchmark Models

1. Linear Models (Logistic Regression):

- **Use Case:** Binary relevance prediction (e.g., likelihood of clicking a video).
- **Advantages:** Fast to train, interpretable, and easy to implement.
- **Limitations:** Assumes linear relationships $\downarrow$, which may oversimplify user-video interactions.

7. Graph Neural Networks (GNNs):

- **Use Case:** Model relationships between users, videos, and categories as a graph.
- **Advantages:** Captures complex interactions in a multi-entity network.
- **Limitations:** High computational overhead for large graphs.

2. Model Comparison and Selection

Model	Strengths	Weaknesses
Logistic Regression	Simple, interpretable, and fast for benchmarking.	Limited in capturing non-linear patterns.
Matrix Factorization	Effective for collaborative filtering tasks.	Cold start issues, no feature learning.
XGBoost	Excellent for structured data with strong feature interactions.	Expensive training for large datasets.
DNNs	Versatile, captures non-linear relationships.	Computationally intensive.
Sequence Models	Ideal for sequential data (e.g., watch history).	Requires large-scale resources.
Two-Tower Model	Highly scalable and efficient for embedding-based retrieval.	Relies on high-quality embeddings.
GNNs	Powerful for multi-entity relationships.	Overhead for very large graphs.

3. Final Model Choices

Benchmark Model: Logistic Regression

- **Reason:** Provides a baseline to measure performance against more complex models.
- **Goal:** Predict whether a user will click a recommended video (binary classification).

4. Training the Model

Data Preparation:

1. Feature Encoding:

- User features (e.g., watch history, demographics) and video features (e.g., metadata, engagement metrics) are converted to embeddings.

2. Negative Sampling:

- For recommendations, randomly sample videos the user did not interact with to create balanced training data.

Training Steps:

1. Embedding Training:

- Train user and video embeddings jointly using labeled interaction data (e.g., click = 1, no click = 0).

2. Loss Function:

- **Binary Cross-Entropy Loss** for click prediction or **Ranking Loss** (e.g., Triplet Loss) to optimize video rankings.

3. Optimization:

- Use SGD or Adam with learning rate scheduling.

Infrastructure:

- Distribute training across GPUs/TPUs using frameworks like TensorFlow or PyTorch.
- Leverage large-batch training for efficient processing of the dataset.

5. Evaluation Metrics

Offline Evaluation:

- **Precision@K / Recall@K**: Measures the accuracy of the top-K recommendations.
- **Normalized Discounted Cumulative Gain (NDCG)**: Evaluates the ranking quality of recommended videos.
- **ROC-AUC**: Checks overall classification performance for click prediction.

Online Evaluation (A/B Testing):

- **Click-Through Rate (CTR)**: Tracks how often users click on recommended videos.
- **Average Watch Time**: Measures user engagement.
- **Diversity Score**: Ensures varied recommendations to prevent monotony.
- **Retention Rate**: Measures whether users return to the platform after watching recommended content.

實際操作 : Chatgpt 練習 - Prompt Framework

Deployment and Monitoring

Prompt 1

Deployment and Monitoring. Ask questions to clarify the resources we have on deployment

Prompt 2

Notes & Feedbacks from previous section +

Once the model shows satisfactory performance, deploy it in a production environment where it can start recommending replacements to users. Monitor the model's performance over time to catch any degradation or shifts in data distribution.

Problem
Understanding

Business
Goals

Data Pipeline

Feature
Engineering

Model Development
and Evaluation

Deployment

Deployment and Monitoring. Ask questions to clarify the resources we have on deployment

Questions to Clarify Deployment Resources and Requirements

Before detailing the deployment and monitoring strategy, I would ask the following questions to understand the available resources, constraints, and objectives:

Infrastructure and Environment:

1. **Cloud Platform:** Are we deploying the model on a specific cloud provider (e.g., AWS, GCP, Azure) or on-premise infrastructure?
2. **Compute Resources:**
 - What types of instances are available (e.g., CPUs, GPUs, TPUs)?
 - Are there constraints on scaling (e.g., autoscaling, budget limitations)?
3. **Data Storage and Access:**
 - What storage systems are in place (e.g., S3, HDFS, BigQuery)?
 - Is there a fast-access cache (e.g., Redis) for real-time feature retrieval?

Latency and Throughput:

4. **Latency Requirements:**
 - What is the acceptable response time $\updownarrow$ recommendations (e.g., <100ms)?

We use GCP, we have GPUs

Once the model shows satisfactory performance, deploy it in a production environment where it can start recommending replacements to users. Monitor the model's performance over time to catch any degradation or shifts in data distribution.

< 2/2 >



Deployment and Monitoring Strategy for the YouTube Recommendation System

Given the deployment on **GCP** with GPU support, the system will leverage Google's managed services and infrastructure to deploy, scale, and monitor the model effectively.

1. Deployment Strategy

Model Serving

1. Framework:

- Use **TensorFlow Serving** or **TorchServe** to serve the trained recommendation model. These frameworks are GPU-compatible and allow for high-performance inference.

2. Endpoint Hosting:

- Deploy the model using **Google Cloud AI Platform Prediction**, which supports containerized models with autoscaling for high throughput.

Real-Time Recommendation Workflow

1. Feature Store:

- Store precomputed features (e.g., user $\downarrow$ nbeddings, video embeddings) in **Cloud Bigtable** or **Redis** for low-latency retrieval.

Wrap up with a flowchart

Can you use a markdown to connect the system design components we discussed. Make the graph as detailed as possible
like the format below?

DB

UI <--> User



ML General 練習

- 基本層面
 - 背為主
 - E.g. Precision vs Recall, Supervised learning vs Unsupervised learning
 - Chatgpt 練習
- 著重apply層面
 - 理解應用為主
 - E.g. Given a simple binary classifier
 - Activation($y = wx + b$)
 - what will happen to the number of True Positive if we change the bias to 0?
 - 練習:大學考試
 - <https://www.cs.cmu.edu/~epxing/Class/10701-20/old-exams.html>

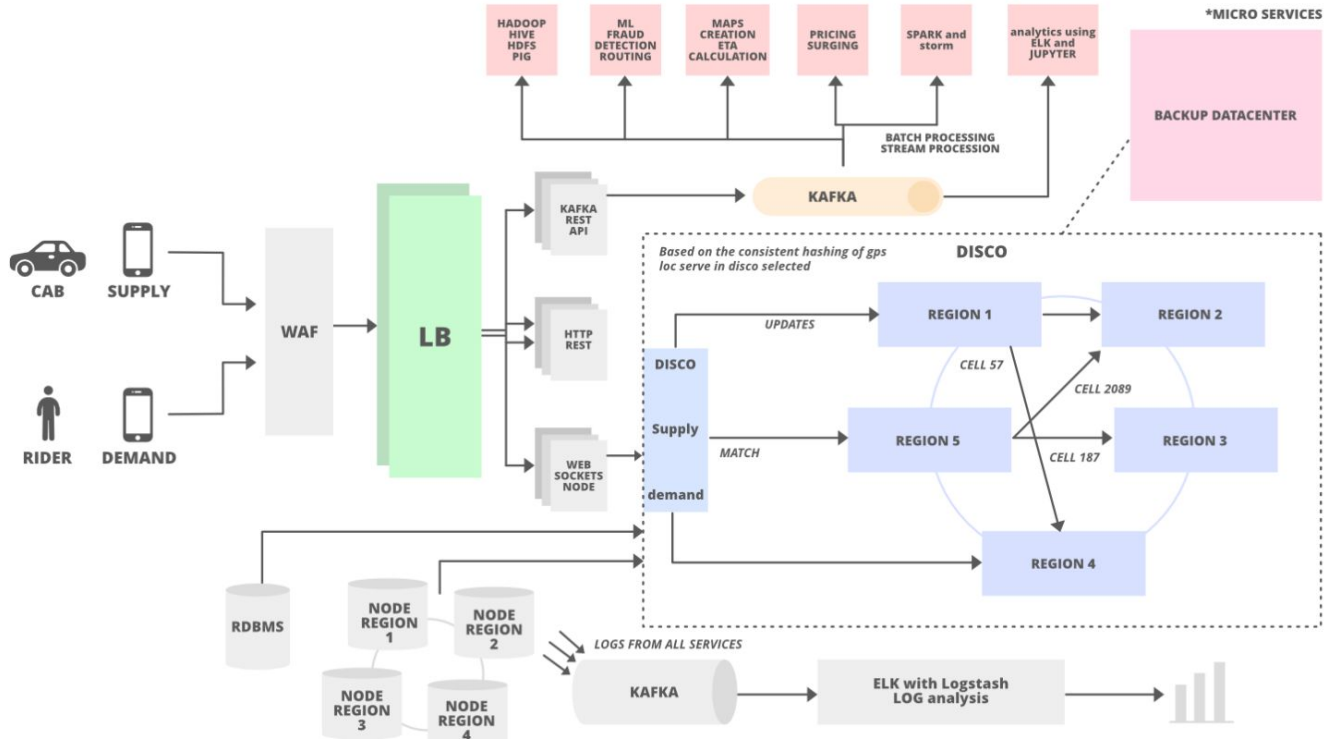
焦點ML System Design 20 Questions

- Design a Recommendation System: Create a scalable recommendation system for e-commerce or streaming platforms like Netflix
- Design a Real-Time Fraud Detection System: Develop a system to detect fraudulent transactions in real time for a payment gateway
- Design an object Classification / Recognition Pipeline: Create an object detection system with Google Map
- Design an Ad Targeting System: Create a system for personalized ad targeting like Google Ads
- Design a Real-Time Notification System: Develop a notification system for push alerts with personalized recommendations
- Design a Spam Detection System: Build a system to detect and filter out spam emails or comments in real-time
- Design a Search Engine: Implement a scalable search engine with machine learning capabilities for relevance ranking
- Design a Predictive Maintenance System: Create a system that predicts equipment failures for manufacturing facilities
- Design a Dynamic Pricing System: Build a system that adjusts prices in real time based on demand, supply, and user behavior
- Design a Conversational Chatbot System: Develop a chatbot capable of providing intelligent responses in customer service scenarios
- Design a Personalized News Feed: Create a scalable news feed system like Facebook's or Twitter's
- Design a Product Recommendation Email System: Build a system that generates and sends personalized email recommendations to users
- Design a Demand Forecasting System: Develop a system for predicting product demand across multiple regions and time periods
- Design a Content Moderation System: Create a system to moderate content (text, image, video) using ML models
- Design a Self-Learning Recommender System: Build a recommendation engine for coupon distribution like Amazon coupons
- Design a Ride-Share Pricing Model: Create a dynamic pricing system for ride-hailing apps like Uber or Lyft
- Design a Real-Time Traffic Prediction System: Develop a system that predicts traffic congestion based on real-time data
- Design a Credit Scoring System: Build a scalable system to assign credit scores based on user financial history and ML models
- Design a Social Media Analytics System: Create a system for analyzing trends, hashtags, and sentiments on social media platforms
- Design a YouTube Home Feed: Build a scalable home feed system for YouTube that integrates personalized recommendations and real-time updates

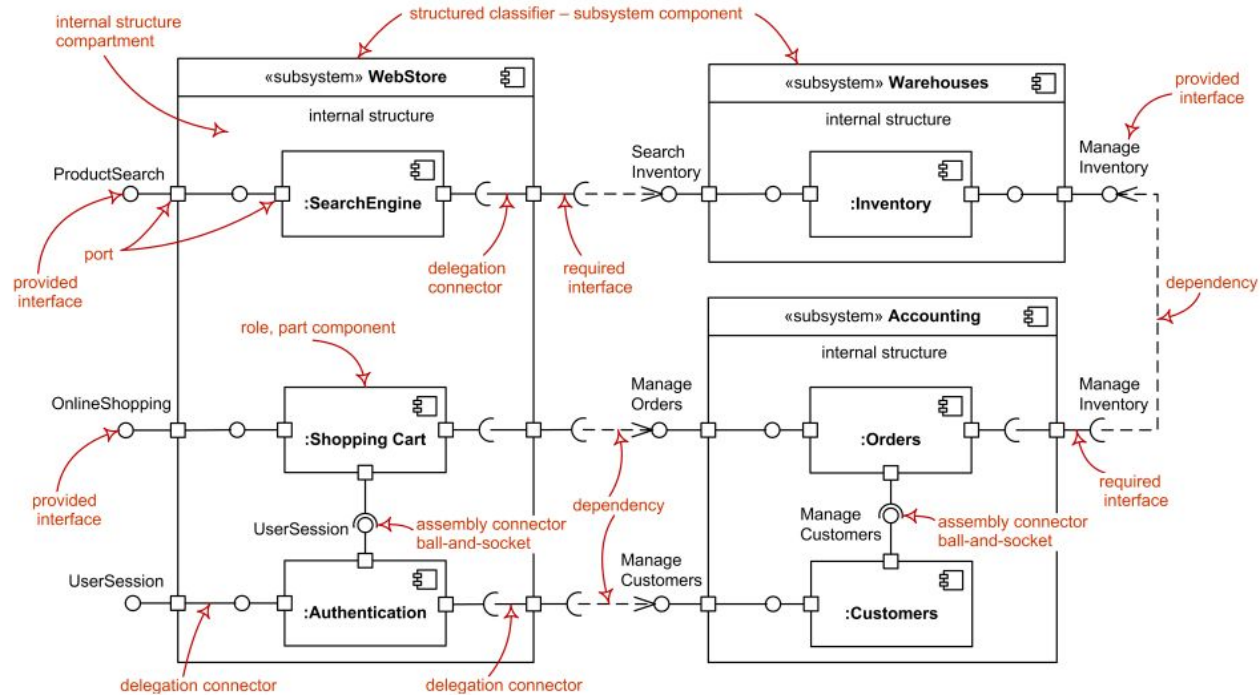
Traditional System Design

簡介

Design a ride-hailing platform

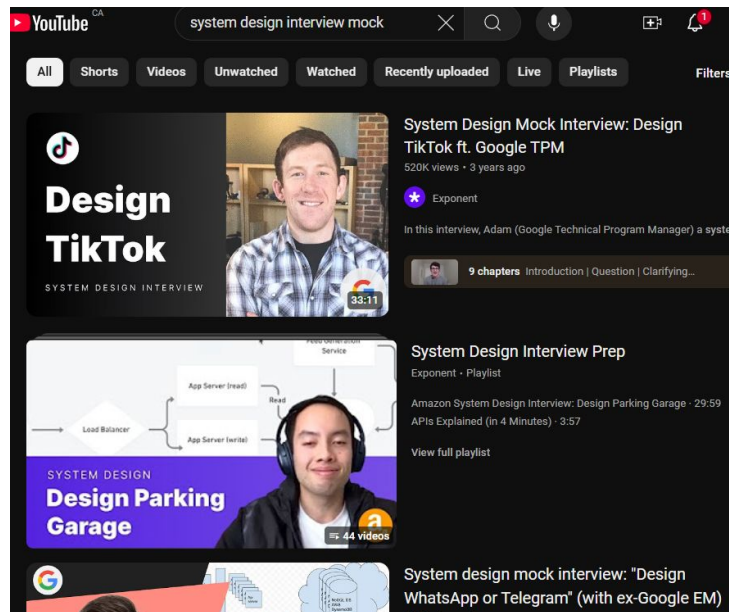
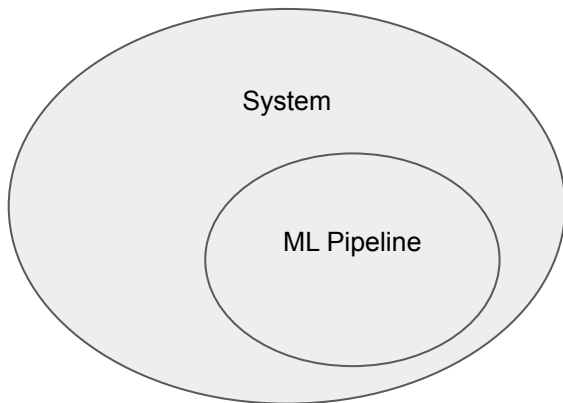


Design an e-commerce platform



Traditional System Design

- 這部分重點在於Identify一個system的key components
 - Based on user requirements → Communication
 - 涉及範圍可以很廣
 - ML pipeline 可以是其中一個component
 - 要清楚每一個component的用處
- Example: https://www.youtube.com/watch?v=_K-eupuDVEc



Major Steps

1. **Problem Understanding**
2. **High-Level Design**
3. **Detailed Design**
4. **Scalability and Optimization**
5. **Testing and Monitoring**
6. **Final Review and Iteration**

Minor Considerations

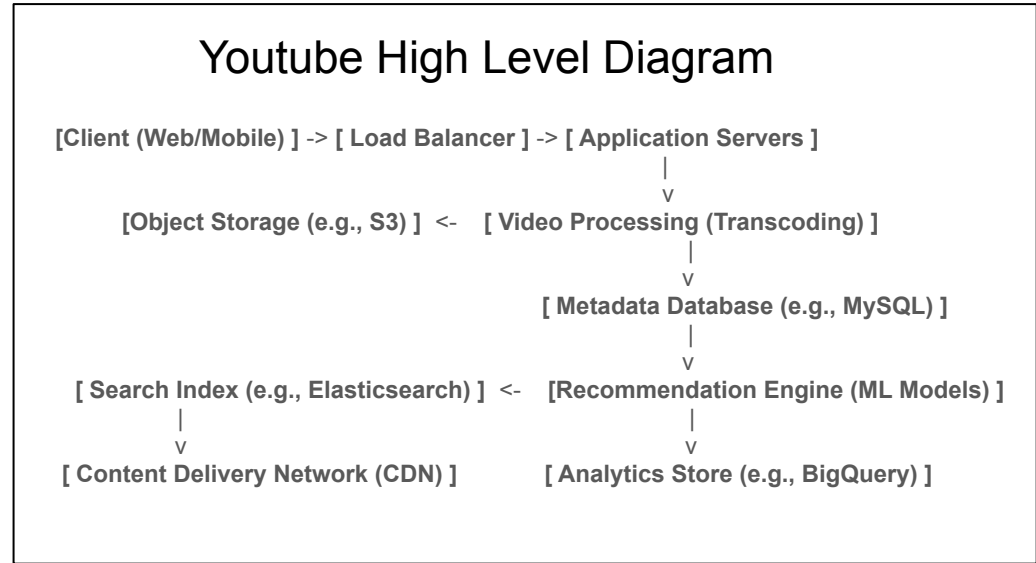
- **Cost Management**
- **Latency Optimization**
- **User Experience (UX)**
- **Security and Privacy**

Major Step 1 - Problem Understanding

- 問問題 - Design Youtube
 - Clarify the Problem Statement & Requirement Gathering
 - Functional Requirements (System的functions)
 - Login, browse and watch video, search
 - Non-functional Requirements (System的behavioral)
 - Response time, security, scalability
- Define Scope
 - 跟Interviewer confirm 背景 / Make Assumptions
 - E.g. Focus on Video Uploading & Search
 - Identify Key Use Cases: Video Uploaded → User Search → Watch
 - Establish Constraints: Determine resource limits (e.g., budget, timeline, infrastructure)
 - Anticipate Challenges: Discuss potential bottlenecks or areas of complexity
- 每步都要Confirm understanding with the interviewer

Major Step 2 - High-Level Design

- Define Core Components
 - Choose Key Technologies
 - E.g. MySQL DB, S3 Bucket
- Create a High-Level Diagram
 - Establish System / Data Flow
- Validate Design



- Client (Web/Mobile): End users interact with YouTube through web or mobile platforms
- Load Balancer: Distributes incoming requests across application servers to ensure high availability
- Application Servers: Handle core business logic for user actions like video uploads, searching, and streaming
- Video Processing: Transcodes uploaded videos into multiple resolutions for adaptive streaming
- Object Storage: Stores the video files and thumbnails (e.g., in AWS S3 or Google Cloud Storage)
- Metadata Database: Stores video metadata (title, description, tags) and user data
- Recommendation Engine: Uses machine learning models to generate personalized recommendations based on user behavior
- Analytics Store: Collects and processes metrics like views, likes, and user engagement for creators
- Search Index: Enables fast and accurate searching of video content and metadata
- Content Delivery Network (CDN): Optimizes video delivery by caching content closer to users globally

Major Step 3 - Detailed Design

- 選擇性深入某幾個components
 - Interview 時間有限
 - 盡量帶Interviewer去你擅長的components
 - 避免被難倒
- 多講Microservices
 - 能show對於題目的understanding

Client (Web/Mobile)

API Gateway:

- A centralized entry point for all client requests.
- Handles user authentication, request validation, and routing.

Load Balancer

Microservices (Application Servers):

- **User Service:** Handles user profiles, playlists, subscriptions, and likes.
- **Video Service:** Manages video upload, storage, playback, and metadata updates.
- **Search Service:** Uses Elasticsearch for fast retrieval of videos based on queries.

Databases:

- **Metadata Database:** Stores structured data like user info, video metadata, and playlists.
- **Search Index:** Powers the search functionality with ranked results.

Video Processing:

- Converts uploaded videos into multiple resolutions for adaptive bitrate streaming.
- Generates thumbnails for UI display.

Object Storage

Recommendation Engine:

- Implements machine learning for personalized content recommendations.
- Uses collaborative and content-based filtering.

Analytics Service

Content Delivery Network (CDN)

Monitoring & Logging

Major Step 4 - Scalability and Optimization

- Based on 已有信息 (From Step 1 Define Scope) 去討論 Optimization
- Scalability refers to the ability of a system to handle increased load by adding resources
 - Vertical scaling (upgrading existing hardware)
 - Replication → Adding replicas on the same server
 - Horizontal scaling (adding more machines)
 - Load Balancing
 - Database Sharding
 - Distributed Systems
 - Asynchronous Processing
- Optimization focuses on improving the system's efficiency to minimize resource usage and latency
 - Caching
 - Indexing
 - Compression
 - Efficient Algorithms
 - Minimizing Network Overhead
- 需要理解 Popular Tools
 - E.g. Distributed Systems
 - Apache Kafka for messaging, Hadoop or Spark for distributed data processing

Major Step 5 - Testing and Monitoring

- Testing
 - Unit Testing
 - Integration Testing
 - Load and Stress Testing
 - End-to-End (E2E) Testing
 - Chaos Testing
 - Regression Testing
 - Security Testing
 - 了解每一種Test 的意義及Tools
- Monitoring
 - 主力想metrics去monitor system
 - System metics: CPU usage, memory, disk I/O, and network traffic
 - Performance metics: Latency, error rates, and transaction times
 - 相對應的Tools
 - System metics: Grafana Dashoard

Major Step 6 - Final Review and Iteration

- 最後一步，通常不太重要，但會比較完整
- Comprehensive Design Review
 - Cross-Team Feedback
 - Checklist-Based Review
- Security and Compliance Audit
 - Verify encryption, authentication, and access control mechanisms
- Documentation and Knowledge Sharing
- Feedback Loop
 - Stakeholder Feedback
 - Iterative Refinement
- Deployment
 - Deployment Planning
 - Rollback Strategy

最有效練習 : Chatgpt Building the System

實際操作 : Chatgpt 練習 - Prompt Framework

Step 1: Requirements Gathering

Let's say now we are in a software engineer interview, I'll give you a question of system design.

*The question is: **Design YouTube**.*

After you receive the question, do not start to answer, I'll prompt you step by step.

For each step, give me all the details and explanations as much as possible.

First of all, before jumping into any solution, start by asking some questions to understand the problem (keep this clean, not too many questions) and understand the scale and size of the problem. And also the system requirements.

During Interview: Expect 去問 Interviewer 問題, Clarify 題目, Focuses

- 代入 Interviewer 嘗試不同 Requirements
- Jot down the notes/feedbacks for next part

Let's say now we are in a software engineer interview, I'll give you a question of system design.

The question is: Design YouTube.

After you receive the question, do not start to answer, I'll prompt you step by step.

For each step, give me all the details and explanations as much as possible.

First of all, before jumping into any solution, start by asking some questions to understand the problem (keep this clean, not too many questions) and understand the scale and size of the problem. And also the system requirements.



Questions to Understand the Problem

Clarifying Requirements:

1. Core Features:

- What are the key functionalities to prioritize for the design? (e.g., video upload, video playback, search, recommendations, live streaming, comments, subscriptions).
- Should we include both user-facing features (e.g., video suggestions, watch history) and creator-focused tools (e.g., analytics, monetization)?

2. User Roles:

- Are we designing for all user roles (e.g., viewers, creators, admins), or should we focus on a subset?

3. Content Management:

- Are there requirements for content moderation, copyright enforcement, and community guidelines?

Scale and Size:

4. Traffic and Usage:

- What is the expected number of daily active users?
- How many concurrent users should the system support?

5. Content Volume:

- How many videos are expected to be uploaded daily?
- What is the average size and duration of uploaded videos?

6. Data Retention:

- How long should videos and user data (e.g., history, preferences) be retained?

7. Global Reach:

- Should the system support users worldwide with multi-language support and regional regulations (e.g., GDPR, COPPA)?

Non-Functional Requirements:

8. Performance:

- What are the latency requirements for video playback and search results?

9. Scalability:

- Should the system handle future growth in users and content seamlessly?

10. Availability:

- What is the expected uptime (e.g., 99.9%, 99.99%)?

11. Security:

- Are there specific requirements for securing user data, preventing unauthorized access, or mitigating attacks (e.g., DDoS)?

Design Constraints:

12. Technology Stack:

- Are there any preferences or constraints regarding the technology stack?

13. Budget:

- Are there cost limitations for storage, compute resources, or network bandwidth?

Major Components Chatgpt 練習 - Prompt Framework

Step 2: High-level Design

High-level System Design

Architectural Diagram: Sketch a basic flow of core components (clients, servers, databases, APIs, etc.)

Component Breakdown: Explain key components and why they're needed, showing how each part contributes to the overall functionality

High-level System Design

Architectural Diagram: Sketch a basic flow of core components (clients, servers, databases, APIs, etc.).

Component Breakdown: Explain key components and why they're needed, showing how each part contributes to the overall functionality.



High-Level System Design for YouTube

Architectural Diagram

Project creator

Here's a textual representation of the architecture flow:

CSS

Copy code

```
[Clients: Viewers, Creators]
|
v
[API Gateway]
|
v
[Load Balancer]
|
+-----+
|               |
v               v
[Application Servers] [CDN Edge Servers]
|                   |
v                   v
[Metadata Database]  [Video Storage (Blob Storage)]
|                   |
v                   |
[Recommendation Engine]
|                   |
v                   |
[Analytics & Logs]
```

Component Breakdown

1. Clients (Viewers, Creators, Admins):

- **Viewers:** Access the platform via web, mobile apps, or smart TVs to watch videos, search, and engage with content.
- **Creators:** Upload videos, manage channels, view analytics, and interact with their audience.
- **Admins:** Oversee content moderation, enforce community guidelines, and monitor system health.

2. API Gateway:

- Acts as the single entry point for all client requests.
- Handles routing, authentication, and request validation.
- Ensures that API endpoints are consistent and secure.

3. Load Balancer:

- Distributes incoming traffic evenly across application servers.
- Ensures high availability and prevents any single server from being overwhelmed.

4. Application Servers:

- Host the core business logic for YouTube.
- Handle user requests such as video uploads, search queries, comment submissions, and subscriptions.
- Scale horizontally to accommodate growing traffic.

5. Content Delivery Network (CDN):

- Optimizes video playback performance by caching video content at geographically distributed edge servers.
- Reduces latency for users by serving videos from the nearest server.

6. Video Storage (Blob Storage):

- Stores uploaded videos in a scalable, durable, and cost-effective manner.
- Uses distributed systems (e.g., AWS S3, Google Cloud Storage) to handle massive amounts of video data.

7. Metadata Database:

- Stores metadata for videos, such as titles, descriptions, tags, thumbnails, and associated user data.

Major Components Chatgpt 練習 - Prompt Framework

Step 3: Detailed Design

Detailed Design

Give me the detailed design of *Clients (Viewers, Creators, Admins)*

重複N次, 所有 Component都要go through

Component Breakdown

- 1. Clients (Viewers, Creators, Admins):**
 - **Viewers:** Access the platform via web, mobile apps, or smart TVs to watch videos, search, and engage with content.
 - **Creators:** Upload videos, manage channels, view analytics, and interact with their audience.
 - **Admins:** Oversee content moderation, enforce community guidelines, and monitor system health.
- 2. API Gateway:**
 - Acts as the single entry point for all client requests.
 - Handles routing, authentication, and request validation.
 - Ensures that API endpoints are consistent and secure.
- 3. Load Balancer:**
 - Distributes incoming traffic evenly across application servers.
 - Ensures high availability and prevents any single server from being overwhelmed.
- 4. Application Servers:**
 - Host the core business logic for YouTube.
 - Handle user requests such as video uploads, search queries, comment submissions, and subscriptions.
 - Scale horizontally to accommodate growing traffic.
- 5. Content Delivery Network (CDN):**
 - Optimizes video playback performance by caching video content at geographically distributed edge servers.
 - Reduces latency for users by serving videos from the nearest server.
- 6. Video Storage (Blob Storage):**
 - Stores uploaded videos in a scalable, durable, and cost-effective manner.
 - Uses distributed systems (e.g., AWS S3, Google Cloud Storage) to handle massive amounts of video data.
- 7. Metadata Database:**
 - Stores metadata for videos, such as titles, descriptions, tags, thumbnails, and associated user data.



Give me the detailed design of
Clients (Viewers, Creators, Admins)



Detailed Design: Clients (Viewers, Creators, Admins)

1. Clients Overview

Clients are the interfaces used by different user roles (Viewers, Creators, Admins) to interact with the YouTube system. Each role has distinct requirements, features, and interactions, which dictate the client design.

1.1 Viewers

Primary Features:

- Browse and search for videos.
- Watch video content.
- Like, comment, and share videos.
- Subscribe to channels and manage playlists.
- Receive personalized recommendations.
- Manage watch history and preferences.

Client Design Details:

1. Frontend Interface:

- **Web (HTML/CSS/JavaScript frameworks like React/Angular):** Responsive and optimized for all browsers.
- **Mobile (Native iOS/Android):** Developed using Swift for iOS and Kotlin/Java for Android. Alternatively, Flutter/React Native for cross-platform consistency.
- **Smart TVs and IoT Devices:** Minimalistic UI with support for remote controls and voice commands.

2. Backend Interactions:

Give me the detailed design of API gateway

Detailed Design: API Gateway

Role of the API Gateway

The API Gateway is a central component in YouTube's architecture that acts as the single entry point for all client requests. It abstracts the backend complexities and ensures secure, efficient, and scalable communication between clients (viewers, creators, admins) and backend services.

Key Responsibilities

1. Request Routing:

- Routes incoming client requests to the appropriate backend services (e.g., video service, metadata service, recommendation service).

2. Authentication and Authorization:

- Verifies user credentials using tokens (e.g., OAuth 2.0, JWT).
- Ensures role-based access control (RBAC) for viewers, creators, and admins.

3. Rate Limiting and Throttling:

- Protects backend services from overload by limiting the number of requests per user or IP.

4. Data Transformation:

- Transforms client-friendly requests into backend-compatible formats and vice versa (e.g., JSON to gRPC, XML to JSON).

5. Load Balancing:

- Distributes incoming traffic to multiple instances of backend services to optimize performance and reliability.

6. Monitoring and Logging:

- Tracks all incoming and outgoing requests for analytics, debugging, and auditing.

7. Security:

- Enforces HTTPS, handles SSL termination, and prevents common attacks (e.g., DDoS, SQL injection).

Architecture of API Gateway

sql

Copy code

[Clients: Viewers, Creators, Admins]

Major Components Chatgpt 練習 - Prompt Framework

Step 4: Scalability and Optimization

Now talk about the Scalability and Optimization, what component we can scale and optimize

Now talk about the Scalability and Optimization, what component we can scale and optimize

Scalability and Optimization for YouTube's System Design

Overview

Scalability ensures the system can handle increased traffic, data volume, and user interactions without degrading performance. Optimization focuses on improving system efficiency, reducing latency, and minimizing resource usage.

Components to Scale and Optimize

1. API Gateway

- **Scaling:**
 - **Horizontal Scaling:** Deploy multiple instances of the API Gateway behind a load balancer to distribute traffic.
 - **Regional Deployment:** Deploy API Gateways in multiple geographical regions to reduce latency for global users.
- **Optimization:**
 - **Caching:** Cache frequent requests (e.g., video metadata) at the API Gateway to reduce load on backend services.
 - **Rate Limiting and Throttling:** Prevent abusive traffic by limiting requests per client or IP.

2. Load Balancer

- **Scaling:**
 - Use a multi-layer load balancing setup:
 - **Global Load Balancer:** Directs traffic to the nearest region.
 - **Local Load Balancer:** Distributes traffic to application servers within a region.
- **Optimization:**
 - Implement sticky sessions only when necessary (e.g., for live streaming) to avoid inefficiencies.

3. Application Servers

- **Scaling:**
 - **Horizontal Scaling:** Add more servers as traffic increases.
 - **Auto-Scaling:** Automatically add or remove instances based on traffic patterns.
- **Optimization:**
 - **Microservices Architecture:** Break down monolithic services into smaller microservices for independent scaling.
 - **Efficient Backend Frameworks:** Use lightweight, high-performance frameworks (e.g., Node.js, Go).

有需要可以獨立解釋

Major Components Chatgpt 練習 - Prompt Framework

Step 5: Testing and Monitoring

We will need to test the system, talk in detail about the testing process

Unit Testing, Integration Testing, Load and Stress Testing, End-to-End (E2E) Testing, Chaos Testing, Regression Testing, Security Testing

And how we monitor the system

Testing and Monitoring the YouTube System

Testing and monitoring ensure the system operates as expected, handles scale, and remains secure. Here's a detailed breakdown of each testing type and the monitoring approach:

1. Unit Testing

Objective:

To verify individual components or functions work as intended.

Scope:

- Isolated testing of small units such as:
 - API Gateway routes.
 - Metadata database queries.
 - Video upload handlers.
 - Recommendation engine algorithms.

Tools and Approaches:

- Use frameworks like JUnit, PyTest, or Mocha depending on the language.
- Mock dependencies to isolate functionality.
- Automate tests in the CI/CD pipeline.

Examples:

- Verify the API Gateway correctly routes requests to the video service.
- Test video transcoding functions with various input formats.

2. Integration Testing

Objective:

To test how different components interact with each other.

Scope:

3. Load and Stress Testing

Objective:

To ensure the system performs under expected and extreme conditions.

Scope:

- Simulate high traffic (e.g., millions of concurrent video streams).
- Test storage systems under heavy write and read loads.

Tools and Approaches:

- Tools: JMeter, Locust, Gatling, or k6.
- Gradually increase traffic to identify bottlenecks.
- Test failure scenarios by exceeding resource limits.

Examples:

- Measure latency during 10 million concurrent video requests.
- Monitor API Gateway response times under heavy traffic.

4. End-to-End (E2E) Testing

Objective:

To validate the entire workflow from start to finish.

Scope:

- Test user flows such as:
 - Video upload and playback.
 - Searching for videos and watching recommendations.

Wrap up with a flowchart

We will need to test the system, talk in detail about the testing process

Unit Testing, Integration Testing, Load and Stress Testing, End-to-End (E2E) Testing, Chaos Testing, Regression Testing, Security Testing

And how we monitor the system

Can you use a markdown to connect the system design components we discussed? Make the graph as detailed as possible like below format?

```
DB
\
UI <--> User
```

Here's a detailed markdown representation connecting the system design components we discussed:

plaintext

Copy code

```
graph TD
    subgraph Clients
        Viewers
        subgraph Viewers
            WebApp[Web App]
            subgraph WebApp
                APIGateway[API Gateway]
                subgraph APIGateway
                    MetadataService[Metadata Service]
                    subgraph MetadataService
                        MetadataDB[Metadata DB]
                    end
                    RecommendationService[Recommendation Service]
                    MLModels[ML Models]
                    LogsAnalytics[Logs & Analytics]
                    VideoService[Video Service]
                    subgraph VideoService
                        CDN[CDN]
                        BlobStorage[Blob Storage]
                    end
                    UserService[User Service]
                    subgraph UserService
                        AuthService[Auth Service]
                    end
                end
            end
            MobileApp[Mobile App]
            subgraph MobileApp
                APIGateway2[API Gateway (same as Web App flow)]
            end
            SmartTVApp[Smart TV App]
            subgraph SmartTVApp
                APIGateway3[API Gateway (same as Web App flow)]
            end
        end
        Creators
        subgraph Creators
            WebCreatorStudio[Web Creator Studio]
            subgraph WebCreatorStudio
                APIGateway4[API Gateway]
                subgraph APIGateway4
                    VideoUploadService[Video Upload Service]
                    subgraph VideoUploadService
                        VideoProcessingPipeline[Video Processing Pipeline]
                        BlobStorage2[Blob Storage]
                    end
                    MetadataService2[Metadata Service]
                end
            end
        end
    end
```

Major Components Chatgpt 練習 - Prompt Framework

Final Review and Iteration

Final Review and Iteration

- *Security and Privacy*
- *Monitoring and Logging*
- *Fault Tolerance and Disaster Recovery*
- *Trade-offs and Optimization*

先Create完整System Chart 再做Final Review
這部分 Interview時 = Future Work , 可以輕鬆帶過

Final Review and Iteration

To finalize the system design for YouTube, we focus on enhancing **Security and Privacy**, ensuring robust **Monitoring and Logging**, building resilience through **Fault Tolerance and Disaster Recovery**, and addressing **Trade-offs and Optimization** for scalability and performance.

1. Security and Privacy

Key Security Measures

- 1. Authentication and Authorization:**
 - Use OAuth 2.0 for secure token-based user authentication.
 - Implement Role-Based Access Control (RBAC) to differentiate between viewers, creators, and admins.
- 2. Data Encryption:**
 - Encrypt sensitive data (e.g., user credentials, video metadata) both in transit (TLS/SSL) and at rest (AES-256).
 - Use secure communication channels (e.g., HTTPS).
- 3. Content Moderation and Compliance:**
 - Deploy automated tools (e.g., ML models) for detecting inappropriate or copyrighted content.
 - Ensure compliance with regulations like GDPR, COPPA, and regional privacy laws.
- 4. Protection Against Attacks:**
 - Implement Web Application Firewalls (WAF) to block SQL injection, XSS, and CSRF attacks.
 - Use DDoS protection tools (e.g., Cloudflare, AWS Shield).
- 5. Access Control for Services:**
 - Secure APIs using rate limiting and IP whitelisting.
 - Regularly rotate service credentials and enforce multi-factor authentication (MFA) for admins.

焦點System Design 20 Questions

- Design a URL Shortener (like TinyURL) → Considerations: Database schema, encoding mechanism, scalability
- Design a Messaging Service (like WhatsApp or Slack) → Considerations: Message delivery, read receipts, group chats
- Design a Distributed Cache System → Considerations: Cache eviction policies, consistency, and replication
- Design a Search Engine → Considerations: Web crawling, indexing, ranking algorithms
- Design a Social Media Feed (like Twitter or Facebook) → Considerations: Ranking posts, handling millions of users, caching
- Design a Ride-Sharing Service (like Uber or Lyft) → Considerations: Matching drivers to riders, real-time updates, scaling
- Design an E-commerce Website (like Amazon) → Considerations: Product catalog, shopping cart, payment system
- Design a Video Streaming Service (like YouTube or Netflix) → Considerations: Content delivery network, video encoding, recommendation systems
- Design a File Storage System (like Google Drive or Dropbox) → Considerations: Data consistency, deduplication, file sharing
- Design a Rate Limiter → Considerations: Preventing abuse, distributed rate limiting, performance
- Design a Notification System → Considerations: Real-time delivery, scheduling, multi-device support
- Design a Logging System → Considerations: Scalability, log retrieval, querying large datasets
- Design a Payment Gateway → Considerations: Security, fraud detection, handling retries
- Design a Recommendation System (for YouTube or Netflix) → Considerations: Collaborative filtering, scalability, personalization
- Design a News Aggregator (like Google News) → Considerations: Content scraping, categorization, ranking
- Design a Metrics Monitoring System (like Prometheus or Datadog) → Considerations: Time-series databases, alerting, scaling
- Design a Content Delivery Network (CDN) → Considerations: Caching, edge servers, load balancing
- Design an Online Multiplayer Game System → Considerations: Low-latency communication, cheat prevention, matchmaking
- Design a Booking System (for Flights, Hotels, or Concerts) → Considerations: Availability, concurrency, search and filters
- Design an API Rate Limiting Service → Considerations: Throttling mechanisms, distributed consistency, monitoring

情報

- [1point3arcs](#)
- <https://www.1point3acres.com/bbs/thread-1097035-1-1.html>